

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО
ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В.
Г. ШУХОВА» (БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

Лабораторная работа №1

по дисциплине: Компьютерные сети
тема: «Протокол сетевого уровня IPX»

Выполнил: ст. группы ПВ-223
Дмитриев Андрей
Александрович

Проверил:
Рубцов Константин Анатольевич

Вариант 2.

Цель работы: изучить протокол сетевого уровня IPX, основные функции API драйвера IPX и разработать программу для приема/передачи данных.

Краткие теоретические сведения

Протокол IPX – это протокол сетевого уровня модели взаимодействия открытых систем (OSI) реализующий передачу пакетов (сообщений) между станциями сети на уровне датаграмм.

Датаграмма – это сообщение, доставка которого получателю не гарантируется.

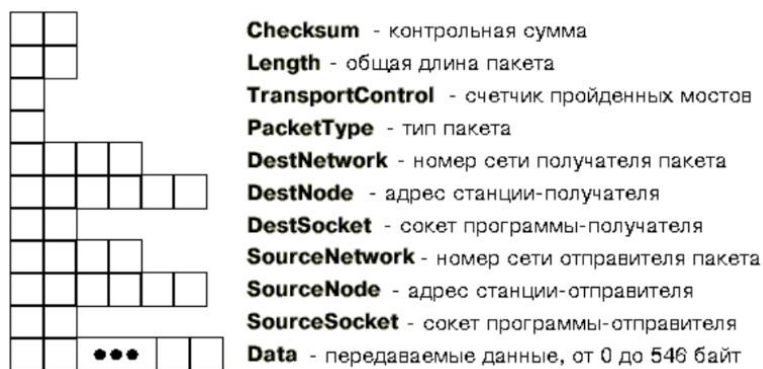
В процессе обмена сообщениями на уровне сеанса связи участвуют только две станции сети. На уровне датаграмм есть возможность посылать сообщение одновременно всем станциям сети.

Номер сети - это номер сегмента сети, определяемого системным администратором. Если в общей сети есть мосты, каждая отдельная сеть, подключенная через мост, должна иметь свой, уникальный номер сети.

Адрес станции - это число, которое является уникальным для каждой рабочей станции. При использовании адаптеров Ethernet уникальность обеспечивается изготовителем сетевого адаптера.

Идентификатор программы на рабочей станции (сокеты) - число, которое используется для адресации определенной программы, работающей на компьютере.

Формат IPX:



Поле **Checksum** предназначено для хранения контрольной суммы передаваемых пакетов. При формировании пакетов не нужно заботиться о содержимом этого поля, так как проверка данных по контрольной сумме выполняется драйвером сетевого адаптера.

Поле **Length** определяет общий размер пакета. Длина заголовка фиксирована и равна 30 байт. Размер передаваемых в поле Data данных может составлять от 0 до 546 байт. При формировании пакетов не нужно проставлять длину пакета в поле Length, протокол IPX делает это сам.

Поле **TransportControl** является счетчиком мостов, которые проходит пакет на своем пути от передающей станции к принимающей.

Каждый раз, когда пакет проходит через мост, значение этого счетчика увеличивается на единицу. IPX перед передачей пакета сбрасывает содержимое этого поля в нуль. Так как IPX сам следит за содержимым этого поля, при формировании пакетов не нужно изменять или устанавливать его в какое-либо состояние.

Поле **PacketType** определяет тип передаваемого пакета. Для IPX следует установить значение равное 4.

Поле **DestNetwork** определяет номер сети, в которую передается пакет.

Поле **DestNode** определяет адрес рабочей станции, которой предназначен пакет.

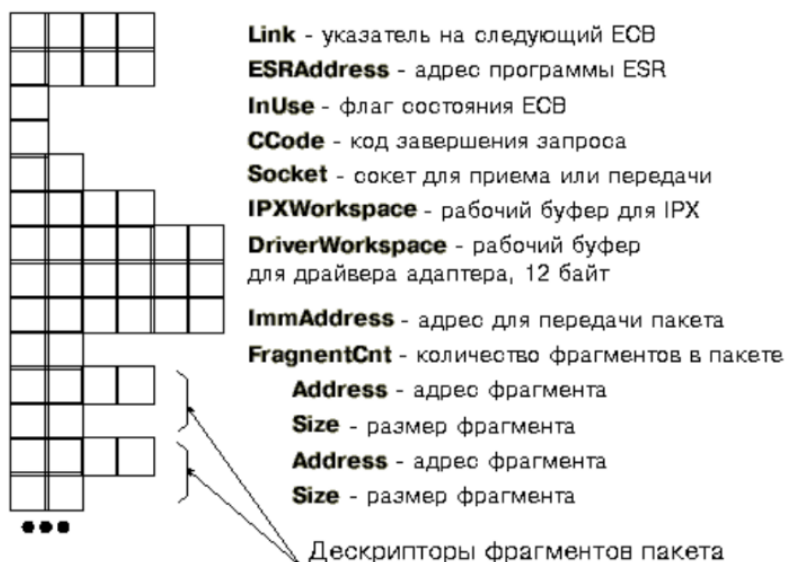
Поле **DestSocket** предназначено для адресации программы, запущенной на рабочей станции, которая должна принять пакет.

Поля **SourceNetwork**, **SourceNode** и **SourceSocket** содержат соответственно номер сети, из которой посылается пакет, адрес передающей станции и сокет программы, передающей пакет.

Поле **Data** содержит передаваемые данные.

Для приема или передачи пакета прикладная программа должна подготовить пакет данных, сформировав его заголовок, и построить так называемый блок управления событием **ECB** (Event Control Block). В блоке ECB задается адресная информация для передачи пакета, адрес передаваемого пакета в оперативной памяти и некоторая другая информация.

Формат блока ECB:



Поле **ESRAddress** содержит полный адрес программного модуля (в формате [сегмент: смещение]), который получает управление при завершении процесса чтения или передачи пакета IPX.

Поле **InUse** может служить индикатором завершения операции приема или передачи пакета.

Результат выполнения можно получить в поле **CCode**, где после выполнения функции IPX содержится код результата выполнения.

Поле **Socket** содержит номер сокета, связанный с данным ECB.

Поле **IPXWorkspace** зарезервировано для использования драйвером IPX. Приложение не должно инициализировать или изменять содержимое этого поля, пока обработка ЕСВ не завершена.

Поле **DriverWorkspace** зарезервировано для использования драйвером сетевого адаптера. Программа не должна инициализировать или изменять содержимое этого поля, пока обработка ЕСВ не завершена.

Поле **ImmAddress** (Immediate Address - непосредственный адрес) содержит адрес узла в сети, в который будет направлен пакет.

Поле **FragmentCnt** содержит количество фрагментов, на которые нужно разбить принятый пакет, или из которых необходимо собрать передаваемый пакет.

Далее располагаются дескрипторы фрагментов, состоящие из указателя в формате [сегмент : смещение] на фрагмент **Address** и поля размера фрагмента **Size**.

Используемые функции API для работы с протоколом IPX

Функция **IPXCloseSocket** предназначена для освобождения сокетов, т.к. сокет является ограниченным ресурсом. На входе BX = 0h, DX = Номер закрываемого сокета. На выходе регистры не используются.

Функция **IPXListenForPacket** предназначена для начала процесса приема пакетов данных из сети. На входе BX = 04h, ES:SI = Указатель на заполненный блок ЕСВ. На выходе регистры не используются.

Сразу после вызова функции IPXListenForPackets в поле InUse блока ЕСВ устанавливается значение FEh, которое означает, что для данного блока ЕСВ ожидается прием пакета.

Функция **IPXSendPacket** подготавливает блок ЕСВ и связанный с ним заголовок пакета для передачи пакета по сети. На входе BX = 03h, ES:SI = Указатель на заполненный блок ЕСВ. На выходе регистры не используются.

Функция **IPXRelinquishControl** (BX = 0Ah) выделяет драйверу IPX процессорное время, необходимое для его правильной работы.

Если программа не использует ESR, она, должна в цикле опрашивать поле InUse блока ЕСВ, для которого выполняется ожидание завершения процесса приема или передачи пакета. Однако для правильной работы драйвера IPX в цикл ожидания необходимо вызвать эту функцию.

Изменяем конфигурацию для использования протокол IPX в DosBox.

```
[ipx]
# ipx: Enable ipx over UDP/IP emulation.

ipx=true

core=auto
cputype=auto
cycles=max
cycleup=10
cycledown=20
```

Листинги исходного кода:

IPX.H:

```
#ifndef IPX_H
#define IPX_H

#include <dos.h>

#define IPX_CMD_OPEN_SOCKET          0x00
#define IPX_CMD_CLOSE_SOCKET        0x01
#define IPX_CMD_GET_LOCAL_TARGET    0x02
#define IPX_CMD_SEND_PACKET         0x03
#define IPX_CMD_LISTEN_FOR_PACKET   0x04
#define IPX_CMD_SCHEDULE_IPX_EVENT  0x05
#define IPX_CMD_CANCEL_EVENT        0x06
#define IPX_CMD_GET_INTERVAL_MARKER 0x08
#define IPX_CMD_GET_INTERNETWORK_ADDRESS 0x09
#define IPX_CMD_RELINQUISH_CONTROL  0x0A
#define IPX_CMD_DISCONNECT_FROM_TARGET 0x0B

#define NO_ERRORS                    0
#define ERR_NO_IPX                   1
#define ERR_NO_SPX                   2
#define NO_LOGGED_ON                 3
#define UNKNOWN_ERROR                0xFF

#define SHORT_LIVED                   0
#define LONG_LIVED                   0xFF
#define IPX_DATA_PACKET_MAXSIZE 546

struct IPXSPX_REGS {
    unsigned int ax;
    unsigned int bx;
    unsigned int cx;
    unsigned int dx;
    unsigned int si;
    unsigned int di;
    unsigned int es;
};
```

```

struct IPX_HEADER {
    unsigned int Checksum;
    unsigned int Length;
    unsigned char TransportControl;
    unsigned char PacketType;
    unsigned char DestNetwork[4];
    unsigned char DestNode[6];
    unsigned int DestSocket;
    unsigned char SourceNetwork[4];
    unsigned char SourceNode[6];
    unsigned int SourceSocket;
};

struct ECB {
    void far* Link;
    void far(*ESRAddress)(void);
    unsigned char InUse;
    unsigned char CCode;
    unsigned int Socket;
    unsigned int ConnectionId;
    unsigned int RestOfWorkspace;
    unsigned char DriverWorkspace[12];
    unsigned char ImmAddress[6];
    unsigned int FragmentCnt;
    struct {
        void far* Address;
        unsigned int Size;
    } Packet[2];
};

unsigned IntSwap(unsigned i);
int IPXOpenSocket(int SocketType, unsigned* Socket);
void IPXCloseSocket(unsigned* Socket);
void IPXListenForPacket(struct ECB* RxECB);
void IPXRelinquishControl(void);
void IPXSendPacket(struct ECB* TxECB);
void PrintECB(const struct ECB* ecb);

#endif // IPX_H

```

IPX.C:

```

#include <stdio.h>
#include <stdlib.h>
#include <dos.h>
#include "IPX.H"

unsigned IntSwap(unsigned i) {
    return ((i >> 8) | (i & 0xFF) << 8);
}

int IPXOpenSocket(int socketType, unsigned* socket) {
    union REGS inregs, outregs;

```

```

    struct SREGS segregs;

    inregs.x.bx = IPX_CMD_OPEN_SOCKET;
    inregs.x.dx = IntSwap(*socket);
    inregs.x.ax = socketType;

    int86x(0x7A, &inregs, &outregs, &segregs);

    *socket = IntSwap(outregs.x.dx);
    return outregs.x.ax;
}

void IPXCloseSocket(unsigned* socket) {
    union REGS inregs, outregs;
    struct SREGS segregs;

    inregs.x.bx = IPX_CMD_CLOSE_SOCKET;
    inregs.x.dx = IntSwap(*socket);

    int86x(0x7A, &inregs, &outregs, &segregs);
}

void IPXListenForPacket(struct ECB* rxECB) {
    union REGS inregs, outregs;
    struct SREGS segregs;

    segregs.es = FP_SEG((void far*)rxECB);
    inregs.x.si = FP_OFF((void far*)rxECB);
    inregs.x.bx = IPX_CMD_LISTEN_FOR_PACKET;

    int86x(0x7A, &inregs, &outregs, &segregs);
}

void IPXSendPacket(struct ECB* txECB) {
    union REGS inregs, outregs;
    struct SREGS segregs;

    segregs.es = FP_SEG((void far*)txECB);
    inregs.x.si = FP_OFF((void far*)txECB);
    inregs.x.bx = IPX_CMD_SEND_PACKET;

    int86x(0x7A, &inregs, &outregs, &segregs);
}

void IPXRelinquishControl(void) {
    union REGS inregs, outregs;
    struct SREGS segregs;

    inregs.x.bx = IPX_CMD_RELINQUISH_CONTROL;

    int86x(0x7A, &inregs, &outregs, &segregs);
}

```

```

void PrintECB(const struct ECB* ecb) {
    unsigned int i;
    printf("ECB Structure:\n");
    printf("  Link: %Fp\n", ecb->Link); // %Fp для far-указателя
    printf("  ESRAAddress: %Fp\n", ecb->ESRAAddress); // %Fp для far-указателя
    printf("  InUse: 0x%02X\n", ecb->InUse);
    printf("  CCode: 0x%02X\n", ecb->CCode);
    printf("  Socket: 0x%04X\n", ecb->Socket);
    printf("  ConnectionId: 0x%04X\n", ecb->ConnectionId);
    printf("  RestOfWorkspace: 0x%04X\n", ecb->RestOfWorkspace);

    printf("  DriverWorkspace: ");
    i = 0;
    while(i < 12) {
        printf("%02X ", ecb->DriverWorkspace[i]);
        i++;
    }
    printf("\n");

    printf("  ImmAddress: ");
    i = 0;
    while(i < 6) {
        printf("%02X ", ecb->ImmAddress[i]);
        i++;
    }
    printf("\n");

    printf("  FragmentCnt: %u\n", ecb->FragmentCnt);
    i = 0;
    while (i < 2) {
        printf("  Packet[%d]:\n", i);
        printf("    Address: %Fp\n", ecb->Packet[i].Address);
        printf("    Size: %u\n", ecb->Packet[i].Size);
        i++;
    }
}

```

SERVER.C:

```

#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <mem.h>
#include <string.h>
#include "IPX.H"

#define BUFFER_SIZE 512

unsigned int packets_sent = 0;
unsigned int packets_received = 0;

void main(void) {
    static unsigned Socket = 0x4567;
    struct ECB TxECB;

```



```

struct IPX_HEADER TxHeader;
unsigned char TxBuffer[BUFFER_SIZE];
FILE* file;
size_t bytesRead;
unsigned int packetNumber = 0;

printf("\n*Server IPX*\n\n");

if (IPXOpenSocket(LONG_LIVED, &Socket)) {
    printf("Opening socket error!\n");
    exit(-1);
}

file = fopen(".\\image.jpg", "rb");
if (!file) {
    printf("Failed to open file.\n");
    IPXCloseSocket(&Socket);
    exit(-1);
}

printf("socket: %x swapped=%x\n", Socket, IntSwap(Socket));

memset(&TxECB, 0, sizeof(TxECB));
TxECB.Socket = IntSwap(Socket);
TxECB.FragmentCnt = 2;
TxECB.Packet[0].Address = &TxHeader;
TxECB.Packet[0].Size = sizeof(TxHeader);
TxECB.Packet[1].Address = TxBuffer;
TxECB.Packet[1].Size = BUFFER_SIZE;

memset(TxECB.ImmAddress, 0xff, 6);

TxHeader.PacketType = 4;
memset(TxHeader.DestNetwork, 0, 4);
memset(TxHeader.DestNode, 0xff, 6);
TxHeader.DestSocket = IntSwap(Socket);

while ((bytesRead = fread(TxBuffer, 1, BUFFER_SIZE, file)) > 0) {
    printf("Sending packet:\n");
    PrintECB(&TxECB);

    IPXSendPacket(&TxECB);
    packets_sent++;
    printf("Packet %d sent (%u bytes).\n", packetNumber++, bytesRead);

    {
        struct ECB RxECB;
        struct IPX_HEADER RxHeader;
        unsigned char RxBuffer[BUFFER_SIZE];

        memset(&RxECB, 0, sizeof(RxECB));
        RxECB.Socket = IntSwap(Socket);
    }
}

```

```

    RxECB.FragmentCnt = 2;
    RxECB.Packet[0].Address = &RxHeader;
    RxECB.Packet[0].Size = sizeof(RxHeader);
    RxECB.Packet[1].Address = RxBuffer;
    RxECB.Packet[1].Size = BUFFER_SIZE;

    IPXListenForPacket(&RxECB);

    printf("Waiting for client acknowledgment...\n");
    while (RxECB.InUse) {
        IPXRelinquishControl();
        if (kbhit()) {
            getch();
            RxECB.CCode = 0xfe;
            break;
        }

        delay(30);
    }

    printf("End listen...\n");
    PrintECB(&RxECB);

    if (RxECB.CCode == 0) {
        packets_received++;
        printf("Acknowledgment received.\n");
    }
    else {
        printf("Failed to receive acknowledgment.\n");
        break;
    }
}

fclose(file);
printf("File transfer complete.\n");

printf("\n--- Statistics ---\n");
printf("Packets Sent: %u\n", packets_sent);
printf("Packets Received: %u\n", packets_received);
printf("Packets Lost: %u\n", packets_sent - packets_received);

IPXCloseSocket(&Socket);
exit(0);
}

```

CLIENT.C:

```

#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <mem.h>
#include <string.h>
#include "IPX.H"

```

```

#define BUFFER_SIZE 512

unsigned int packets_received = 0;
unsigned int packets_sent = 0;

void main(void) {
    static unsigned Socket = 0x4567;
    struct ECB RxECB;
    struct IPX_HEADER RxHeader;
    unsigned char RxBuffer[BUFFER_SIZE];
    FILE* file;
    size_t bytesWritten;
    unsigned int packetNumber = 0;

    printf("\n*Client IPX*\n\n");

    if (IPXOpenSocket(LONG_LIVED, &Socket)) {
        printf("Opening socket error.\n");
        exit(-1);
    }

    file = fopen(".\\recimage.jpg", "wb");
    if (!file) {
        printf("Failed to create file.\n");
        IPXCloseSocket(&Socket);
        exit(-1);
    }

    printf("socket: %x swapped=%x\n", Socket, IntSwap(Socket));

    memset(&RxECB, 0, sizeof(RxECB));
    RxECB.Socket = IntSwap(Socket);
    RxECB.FragmentCnt = 2;
    RxECB.Packet[0].Address = &RxHeader;
    RxECB.Packet[0].Size = sizeof(RxHeader);
    RxECB.Packet[1].Address = RxBuffer;
    RxECB.Packet[1].Size = BUFFER_SIZE;

    PrintECB(&RxECB);

    printf("Waiting for file transfer...\n");

    while (1) {
        IPXListenForPacket(&RxECB);
        printf("Listen...\n");

        while (RxECB.InUse) {
            IPXRelinquishControl();
            if (kbhit()) {
                getch();
                RxECB.CCode = 0xfe;
                break;
            }
        }
    }
}

```

```

    }

    delay(30);
}

printf("End listen...\n");
PrintECB(&RxECB);

if (RxECB.CCode == 0) {
    bytesWritten = fwrite(RxBuffer, 1, BUFFER_SIZE, file);
    if (bytesWritten < BUFFER_SIZE) {
        printf("File write error.\n");
        break;
    }

    packets_received++;
    printf("Packet %d received (%u bytes).\n", packetNumber++,
bytesWritten);

    {
        struct ECB TxECB;
        struct IPX_HEADER TxHeader;
        unsigned char TxBuffer[BUFFER_SIZE];

        memset(&TxECB, 0, sizeof(TxECB));
        TxECB.Socket = IntSwap(Socket);
        TxECB.FragmentCnt = 2;
        TxECB.Packet[0].Address = &TxHeader;
        TxECB.Packet[0].Size = sizeof(TxHeader);
        TxECB.Packet[1].Address = TxBuffer;
        TxECB.Packet[1].Size = BUFFER_SIZE;

        TxHeader.PacketType = 4;
        memcpy(TxHeader.DestNode, RxECB.ImmAddress, 6);
        TxHeader.DestSocket = IntSwap(Socket);

        printf("Packet for send:\n");
        PrintECB(&RxECB);

        strcpy(TxBuffer, "ACK");
        IPXSendPacket(&TxECB);
        packets_sent++;
    }
}
else {
    printf("File transfer complete or canceled.\n");
    break;
}
}

fclose(file);
printf("File received successfully.\n");

```

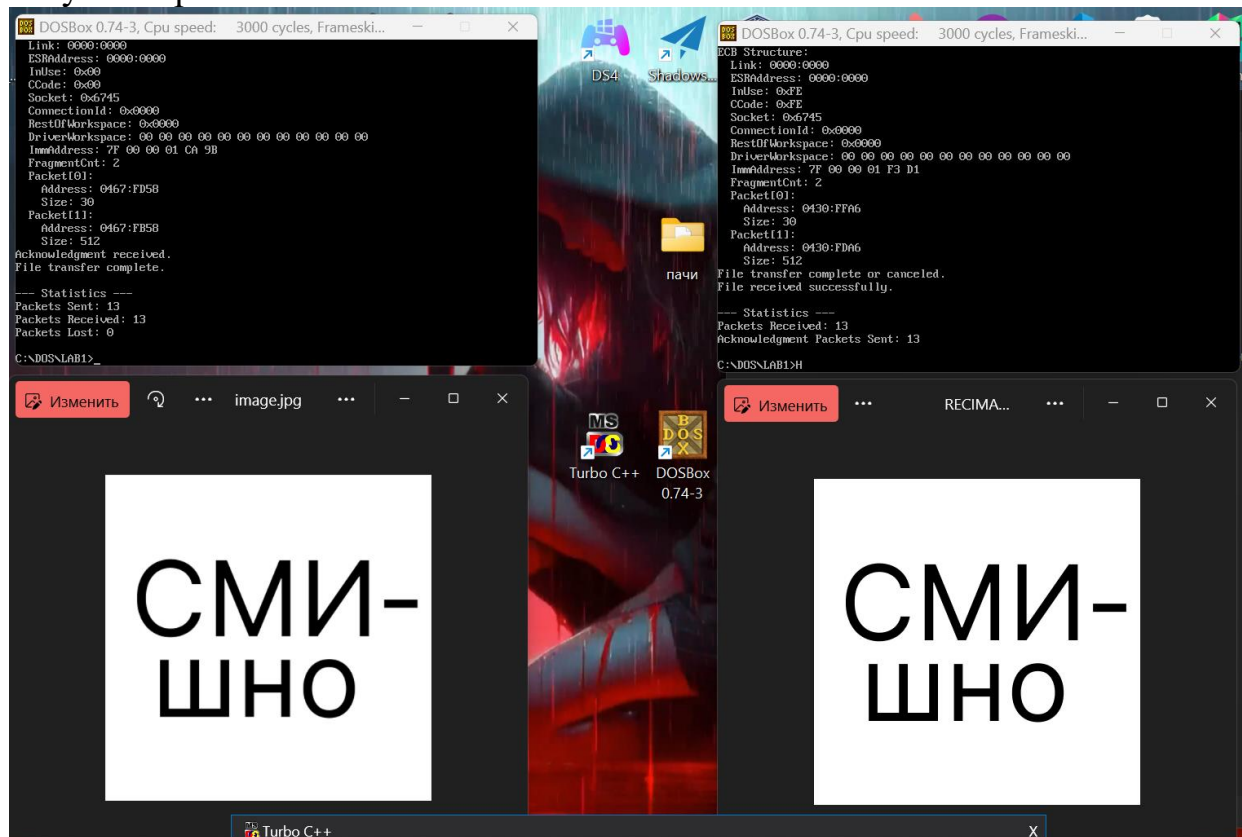
```

printf("\n--- Statistics ---\n");
printf("Packets Received: %u\n", packets_received);
printf("Acknowledgment Packets Sent: %u\n", packets_sent);

IPXCloseSocket(&Socket);
exit(0);
}

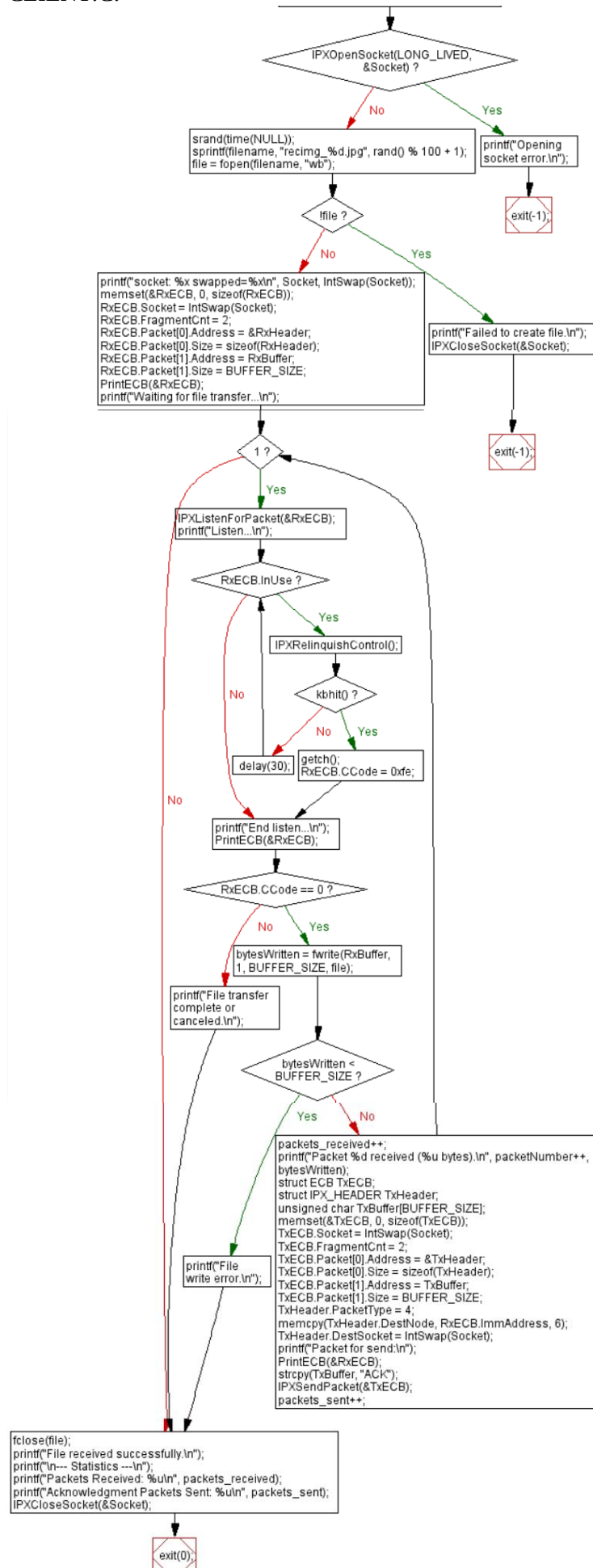
```

Результат работы:

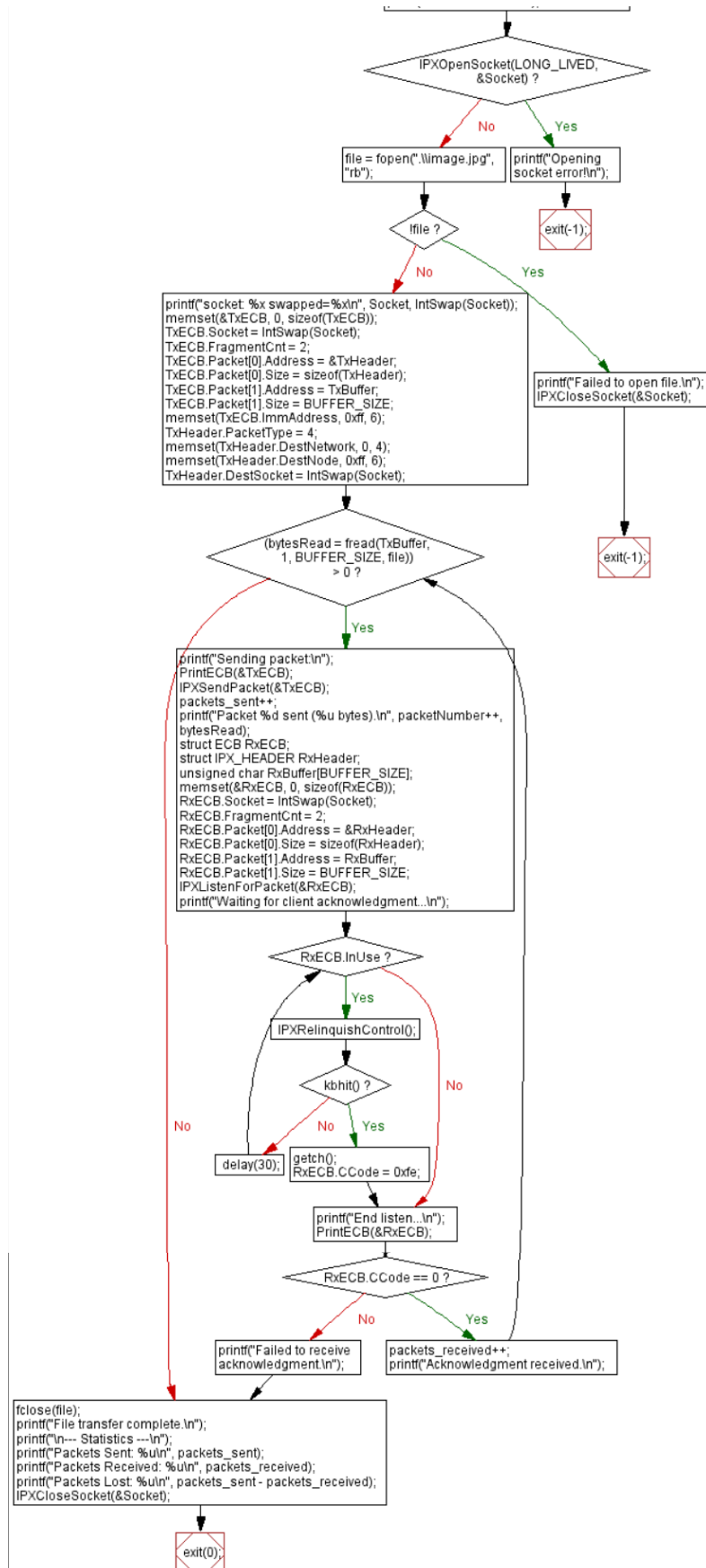


Блок схемы программ:

CLIENT.C:



SERVER.C:



Анализ функционирования разработанных программ

Тест 1.

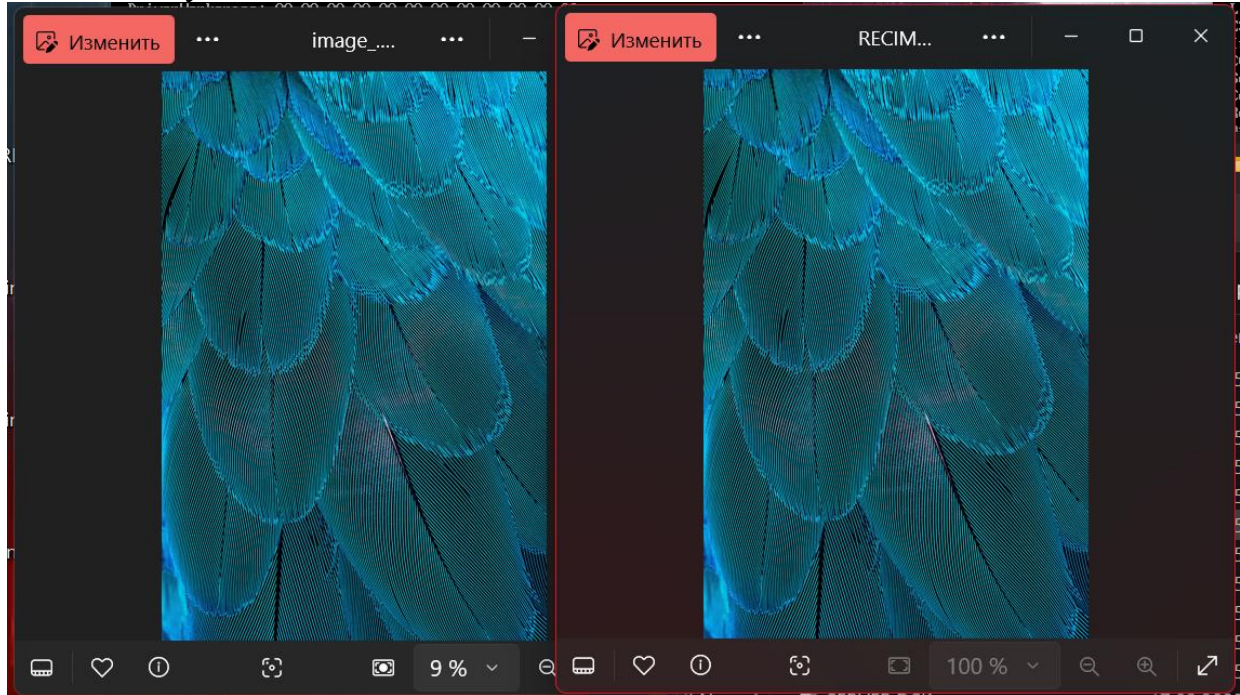
Размер файла: 8451 Кб

Предполагаемое кол-во пакетов по 512байт: 16902

Количество пакетов: 16902 (100 пакетов в 6.8с)

Общее время: ~19мин 9.2с

Кол-во получателей: 1



Тест 2.

Размер файла: 122 Кб

Предполагаемое кол-во пакетов по 512байт: 244

Количество пакетов: 244 (100 пакетов в 6.7с = 60 бит/с)

Общее время: ~17.2с

Кол-во получателей: 1

Общее количество обработанных пакетов у получателя:
244

Тест 3.

Размер файла: 122 Кб

Предполагаемое кол-во пакетов по 512байт: 244

Количество пакетов: 244 (100 пакетов в 6.7с)

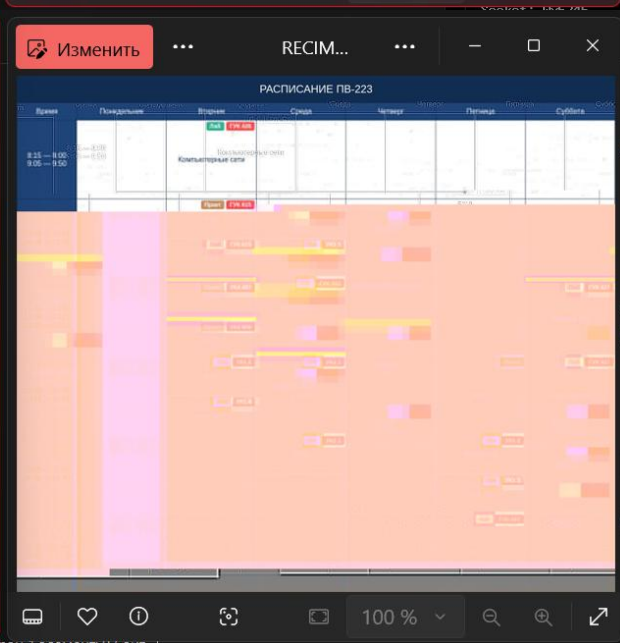
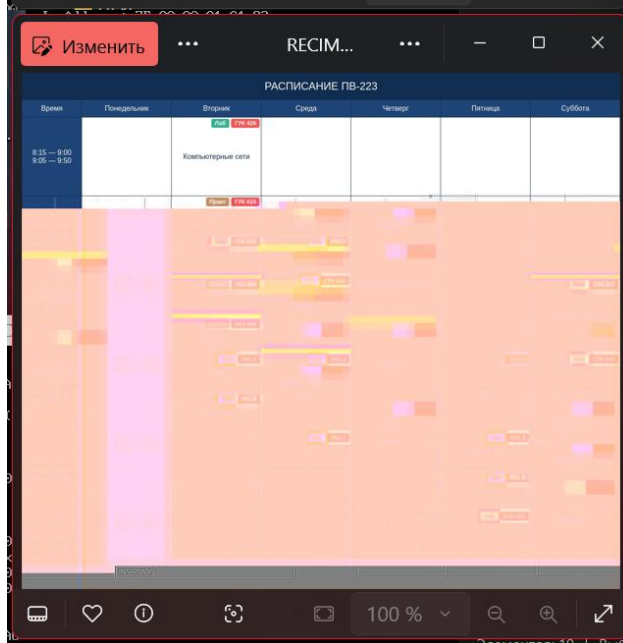
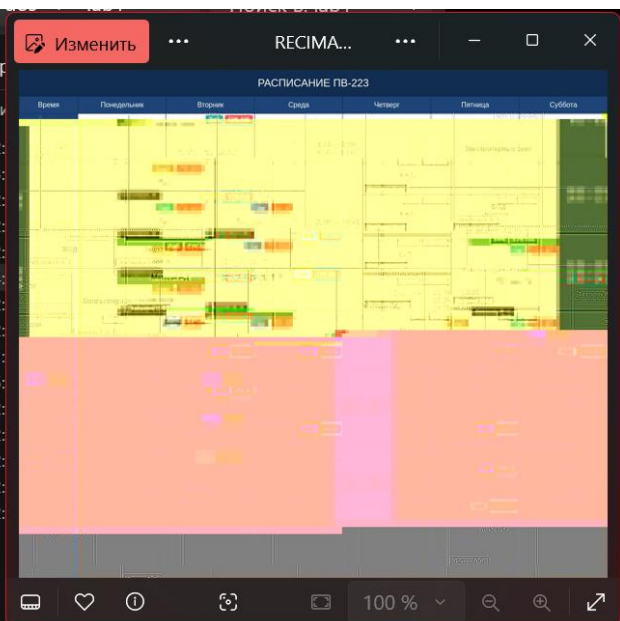
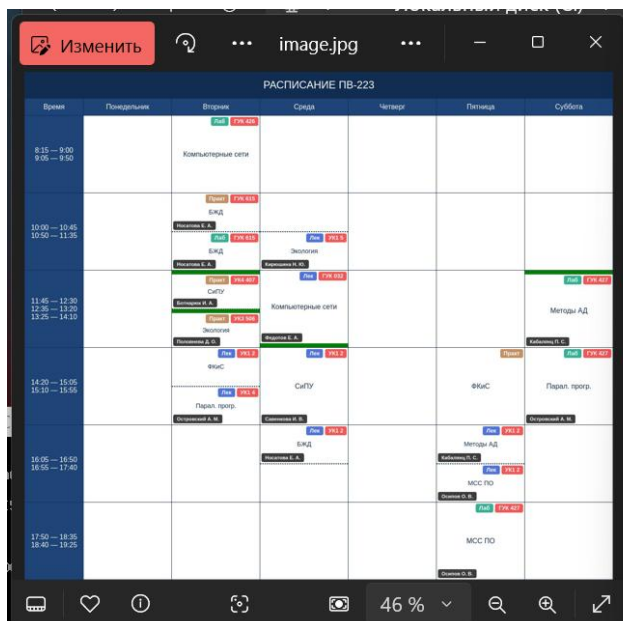
Общее время: ~17.2с

Кол-во получателей: 4

Общее количество обработанных пакетов у получателей:

234	233	232	231
-----	-----	-----	-----

- имеются потери данных:



Вывод: в ходе работы изучены протоколы сетевого уровня IPX, основные функции API драйвера IPX и разработана программа для приема/передачи данных.